

CSE470: SOFTWARE ENGINEERING

# Design Patterns

Observer | Singleton | Adapter | Factory Method

---

Md Nafiu Rahman

Lecturer, Department of Computer Science and Engineering

BRAC University

# What is the Gang of Four (GoF)?

---

1

## Principle 1

Program to an interface,  
not an implementation.

2

## Principle 2

Favor object composition  
over class inheritance.

Gamma, Helm, Johnson, and Vlissides published Design Patterns in 1994, giving software developers a shared vocabulary for reusable object-oriented design.

# Types of design patterns

---



## Creational

How objects are created.  
Singleton and Factory Method.



## Structural

How objects fit together.  
Adapter.



## Behavioral

How objects communicate.  
Observer.

# 01

BEHAVIORAL PATTERN

## The Observer

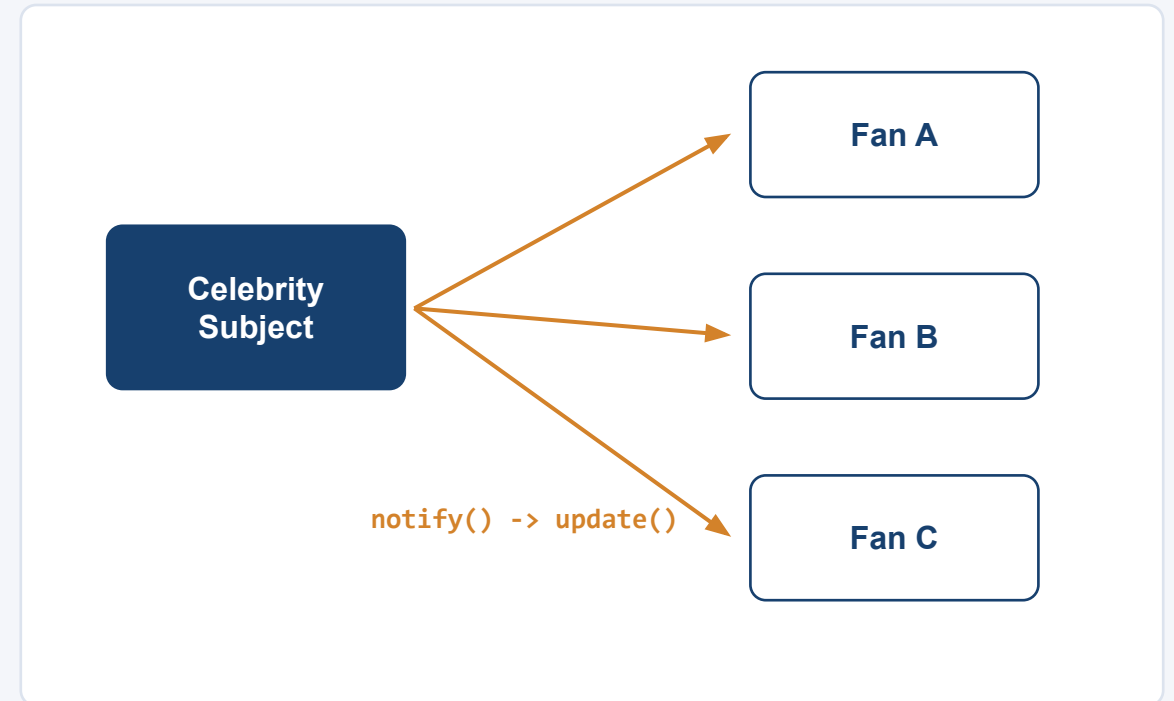
---

One object changes; interested objects are notified automatically.

# Observer Pattern: Intent

## One-to-many dependency

When the Subject changes state, every Observer receives an update. Think of a celebrity posting news and fans receiving it.



# Observer Example: Celebrity and Fan

## Subject (Celebrity)

### SUBJECT

```
class Celebrity:
    def __init__(self):
        self._fans = []
        self._state = None

    def attach(self, fan):
        self._fans.append(fan)

    def notify(self):
        for fan in self._fans:
            fan.update(self)

    def set_state(self, state):
        self._state = state
        self.notify()
```

#### WHAT EACH METHOD DOES

**attach(fan)** adds a fan to the notify list  
**notify()** calls update() on every fan  
**set\_state(s)** saves the state, then notifies

## Observer (Fan)

### OBSERVER

```
class Fan:
    def __init__(self, name):
        self.name = name
        self._celebrities = []

    def update(self, celebrity):
        state = celebrity.get_state()
        print(f"{self.name}: {state}")

    def follow(self, celebrity):
        celebrities.append(celebrity)
        celebrity.attach(self)
```

#### WHAT EACH METHOD DOES

**\_\_init\_\_(name)** stores this fan's name  
**follow(celeb)** registers the fan with the subject  
**update(celeb)** pulls the new state and reacts

# Observer: Test Code

---

— DRIVER

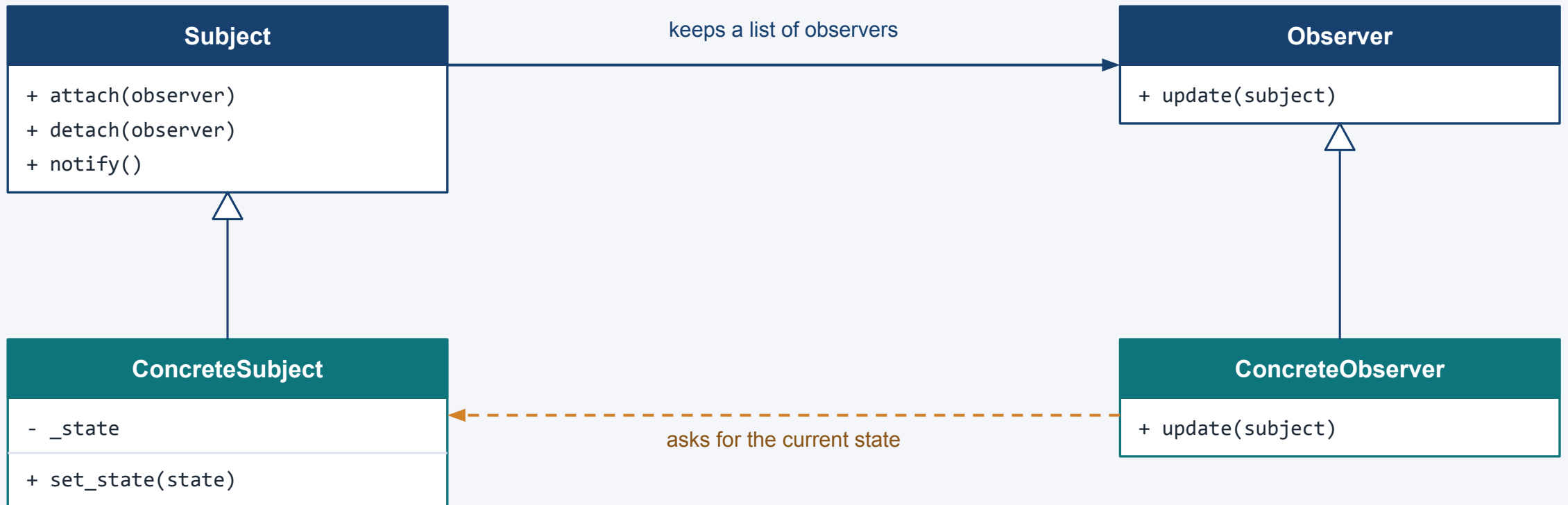
```
celeb = Celebrity()
ryan = Fan("Ryan")
mim = Fan("Mim")

ryan.follow(celeb)
mim.follow(celeb)
celeb.set_state("New film announced")

# Output
# Ryan: New film announced
# Mim: New film announced
```

The subject knows only that it has observers. It does not know what each observer does with the update.

# Observer Pattern: Structure





# 02

CREATIONAL PATTERN

## The Singleton

---

Some resources should have one shared instance for the whole program.

# Singleton Pattern: The Problem

## Problem code

### PROBLEM

```
class HelpDesk:
    def get_service(self):
        print("HelpDesk service")

class Student:
    def service(self):
        desk = HelpDesk()
        desk.get_service()

class Teacher:
    def service(self):
        desk = HelpDesk()
        desk.get_service()
```

Every client makes its own  
HelpDesk object.

For a shared service, configuration, logger,  
cache, or connection pool, multiple instances  
waste resources and can create inconsistent  
state.



# Basic: \_\_new\_\_ versus \_\_init\_\_

## Code example

```
— LIFECYCLE

class MyClass:
    def __new__(cls):
        print("created")
        return super().__new__(cls)

    def __init__(self):
        print("initialized")

obj = MyClass()
```

## \_\_new\_\_ creates the object

- Runs first and decides which object is returned.
- \_\_init\_\_ runs next and initializes that returned object.
- Singleton controls \_\_new\_\_ to reuse an existing instance.

# Singleton Pattern: The Solution

## Singleton code

### SOLUTION

```
class HelpDesk:
    _instance = None

    def __new__(cls):
        if cls._instance is None:
            print("Creating HelpDesk")
            cls._instance = super().__new__(cls)
        return cls._instance

    def get_service(self):
        print("HelpDesk service")
```

## How it works

- 1 Store the only instance in a class variable.
- 2 Create it only when the variable is empty.
- 3 Return the same instance for every later call.

The print inside `__new__` is the proof: it runs on the first call only, never again.

# Singleton: Test Code

```
— DRIVER

client1 = HelpDesk()
client2 = HelpDesk()
client3 = HelpDesk()

client1.get_service()
client2.get_service()
client3.get_service()

print(client1 is client2 is client3)

# Output
# Creating HelpDesk      <- printed once only
# HelpDesk service
# HelpDesk service
# HelpDesk service
# True
```

Three constructor calls, one object: the body of `__new__` ran only the first time.

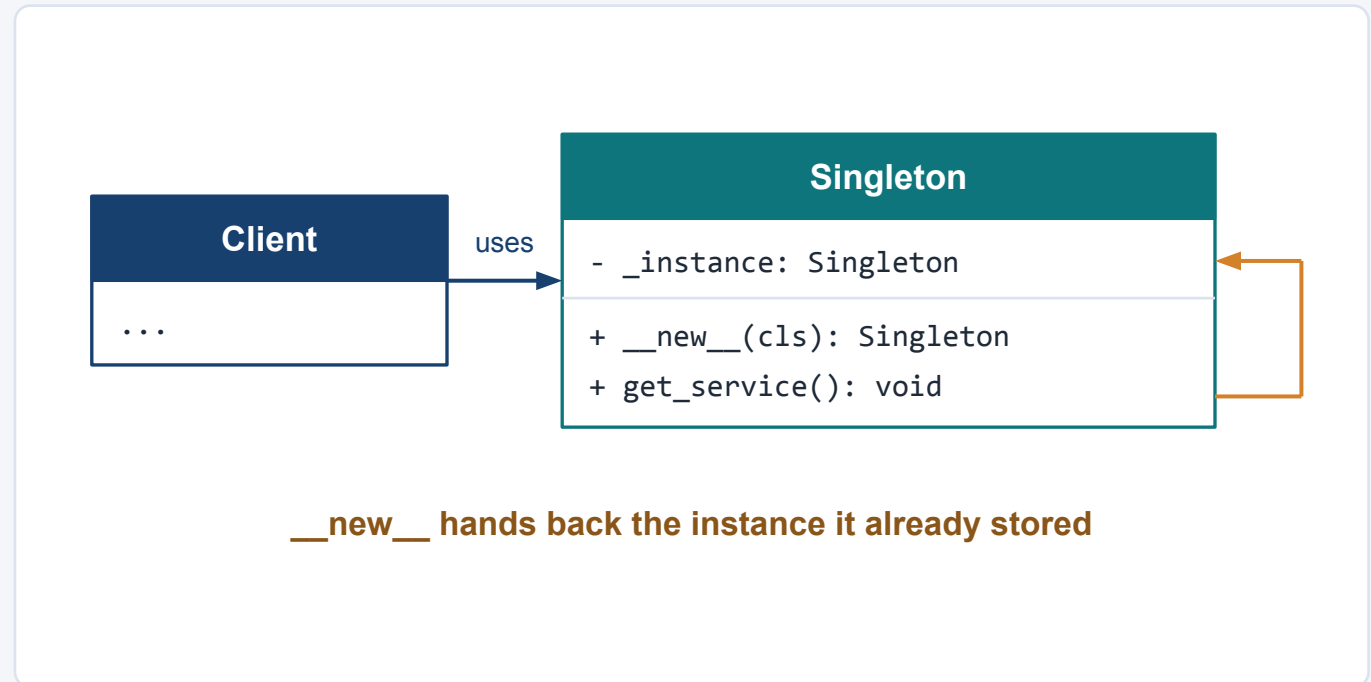
# Singleton Pattern: Structure

## Intent

Ensure a class has one instance and provide a global point of access to it.

## Why lazy initialization?

The object is built on the first request, not when the class is loaded. If nobody ever asks for the HelpDesk, no memory, no database connection, and no config file is ever spent on it. Startup stays fast, and the setup cost is paid only by code that actually uses the object.



# 03

STRUCTURAL PATTERN

## The Adapter

---

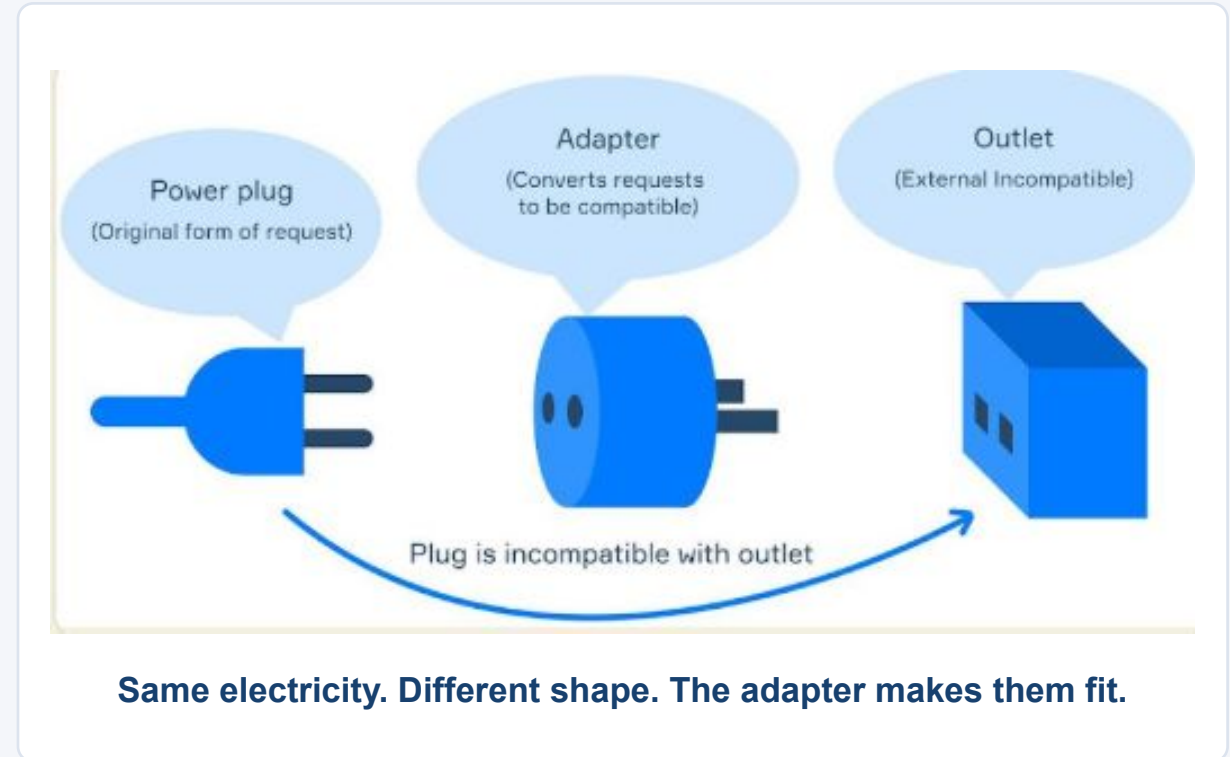
Make two incompatible interfaces work together without rewriting either side.

# Adapter Pattern: Intent

## The wrapper

The plug and the outlet were never designed for each other. Rather than rewire either one, you put a small piece in between that translates.

- **Adaptee** the class you already have
- **Target** the interface the client wants
- **Adapter** sits between and translates





# Adapter Scenario: Pizza

Both objects do the same job. Only the method names disagree.



**TARGET** | what the client expects

**Pizza**

toppings()    buns()



**ADAPTEE** | the class you already have

**DhakaiyaPizza**

dhakaiya\_toppings()    dhakaiya\_buns()

**Same job, different method names, so PizzaStore cannot call it.**

# Adapter Pattern: The Problem

## Client code

### PROBLEM

```
class PizzaStore:
    def serve(self, pizza):
        pizza.toppings()
        pizza.buns()

store = PizzaStore()
store.serve(DhakaiyaPizza())
```

## Adaptee

### WRONG NAMES

```
class DhakaiyaPizza:
    def dhakaiya_toppings(self):
        print("Dhakaiya toppings")

    def dhakaiya_buns(self):
        print("Dhakaiya buns")
```

**AttributeError: the store asks for toppings() and buns(); this class has neither.**

# Object Adapter: Correct Code

## OBJECT ADAPTER

```
class PizzaAdapter(Pizza):  
    def __init__(self, adaptee):  
        self.adaptee = adaptee  
  
    def toppings(self):  
        self.adaptee.dhakaiya_toppings()  
  
    def buns(self):  
        self.adaptee.dhakaiya_buns()
```

## TARGET

```
class Pizza:  
    def toppings(self):  
        pass  
  
    def buns(self):  
        pass
```

The adapter holds the old object and offers the new names. Nothing inside DhakaiyaPizza changes.

# Object Adapter: Test Code

---

```
DRIVER

dhakaiya = DhakaiyaPizza()
pizza = PizzaAdapter(dhakaiya)

pizza.toppings()
pizza.buns()

# Output
# Dhakaiya toppings
# Dhakaiya buns
```

The client only ever sees Pizza. The adapter does the translating.

# Class Adapter: Correct Code

```
— CLASS ADAPTER

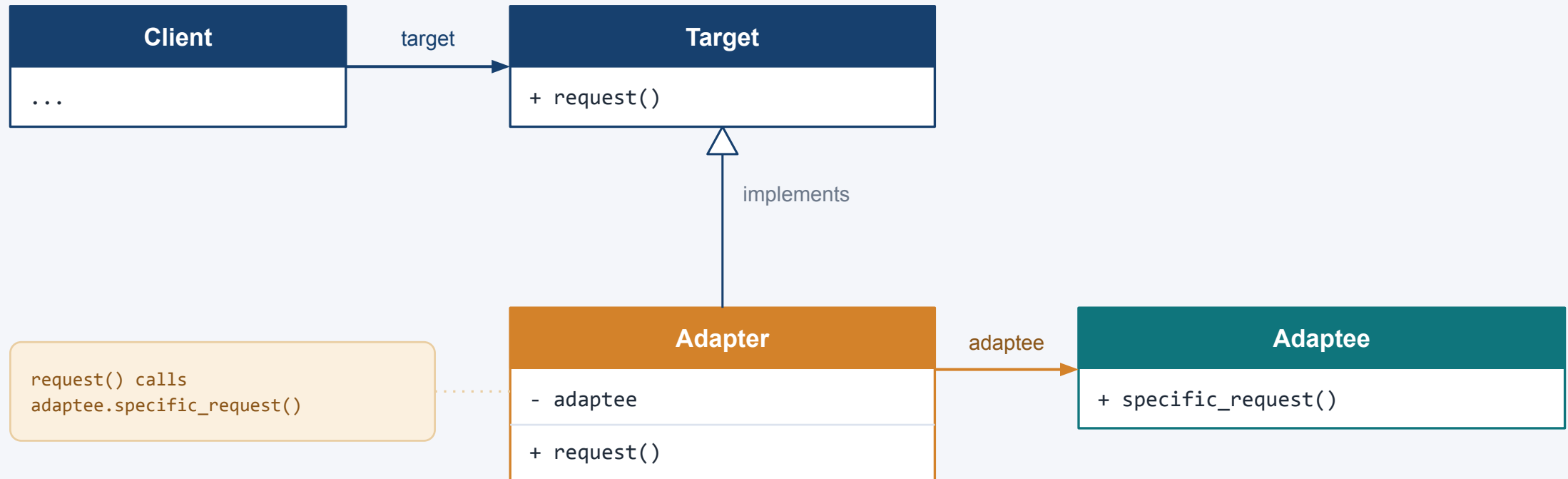
class PizzaClassAdapter(Pizza, DhakaiyaPizza):
    def toppings(self):
        self.dhakaiya_toppings()

    def buns(self):
        self.dhakaiya_buns()

pizza = PizzaClassAdapter()
pizza.toppings()
pizza.buns()
```

Shorter, but locked to one Adaptee class. The object adapter stays more flexible.

# Adapter Pattern: Structure



Target names the method. Adapter implements it. Adaptee already does the work under another name.

# 04

CREATIONAL PATTERN

## Factory Method

---

Let a subclass choose which product object is created.

# Factory Method: The Problem

## Hard-coded product

### PROBLEM

```
class Application:
    def new_document(self):
        doc = DrawingDocument()
        doc.open()
```

Application knows when to create a document, but it should not be tied to one document class.

**Adding TextDocument means editing new\_document().**

## Solution

The parent class knows the steps to run, but not which object to build. So it leaves one hole, a create method, and each subclass fills that hole with the product it wants.

### Use it when

- The steps stay the same, only the object changes.
- You expect to add more product types later.
- Client code should not name concrete classes.



# Factory Method: Correct Code

## CREATOR

```
class Application(ABC):
    def create_document(self): # factory method
        pass

    def new_document(self):
        doc = self.create_document()
        doc.open()

class DrawingApplication(Application):
    def create_document(self):
        return DrawingDocument()

class TextApplication(Application):
    def create_document(self):
        return TextDocument()
```

## PRODUCT

```
class Document:
    def open(self):
        pass

class DrawingDocument(Document):
    def open(self):
        print("Opening drawing document")

class TextDocument(Document):
    def open(self):
        print("Opening text document")
```

**1** Parent owns the workflow.

**2** Factory method makes the product.

**3** Subclass picks the concrete class.

# Factory Method: Client Code

— DRIVER

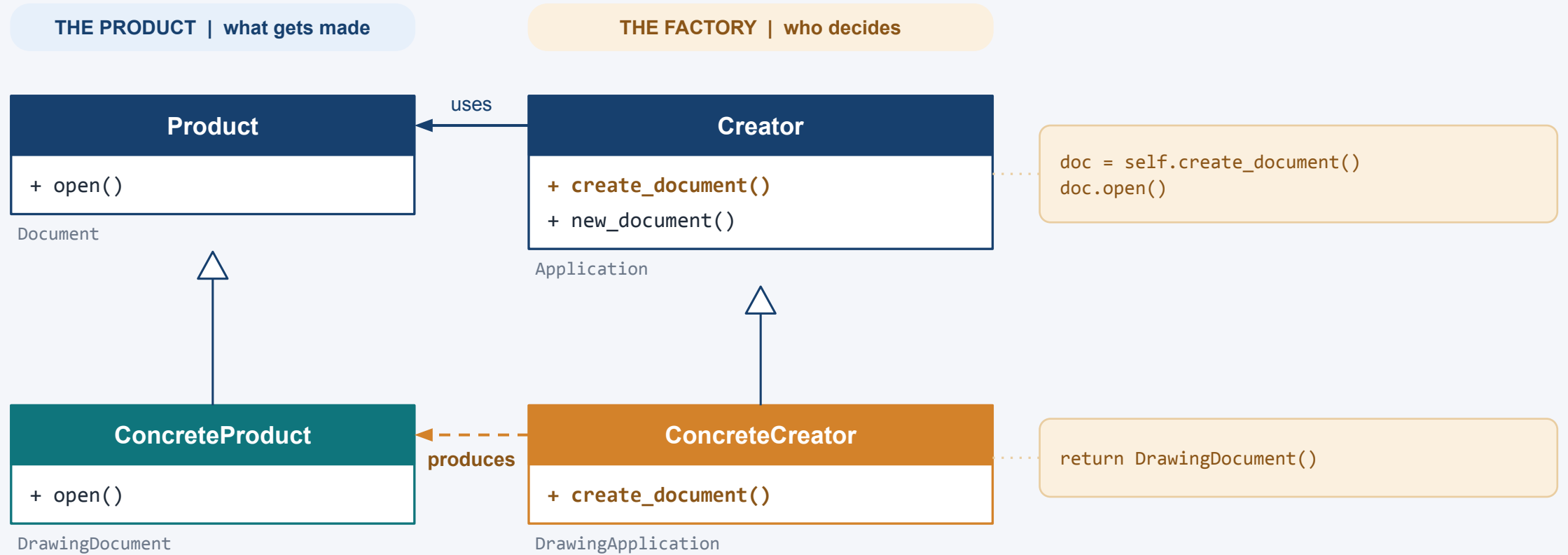
```
app = DrawingApplication()
app.new_document()
# Opening drawing document

app = TextApplication()
app.new_document()
# Opening text document

# Application never named DrawingDocument or
# TextDocument. Changing the subclass changed
# the product; new_document() was not touched.
```

**The client picks the application; the application picks the document.**

# Factory Method: Structure



■ the factory method: the one call each subclass overrides to name its product

# The four patterns side by side

---

## Observer

Many objects react to one change

**Notify** a list behind an interface

## Singleton

One shared resource

Return the **same** instance

## Adapter

Useful class, wrong interface

**Translate** with a wrapper

## Factory Method

Workflow cannot choose a product

Let a **subclass** create it

# Questions?

---

Design patterns are conversations between classes, not blocks of code.